

Prerequisites

Create a
Microsoft
Foundry project

Deploy models

Get the
application files
from GitHub

Create a
speech-
generation app

Create a
speech-
transcription
app

Clean up

Use speech-capable generative AI models

Increasingly, generative AI model capabilities are evolving beyond text-based language completion to support content in other formats - including audible speech.

In this exercise, you'll use generative AI models to support two common scenarios:

- Speech synthesis (text-to-speech) - generating speech output.
- Speech recognition (speech-to-text) - transcribing speech input.

While this exercise is based on Python, you can develop generative AI speech applications using multiple language-specific SDKs; including:

- [OpenAI SDK for Python](#)
- [OpenAI SDK for .NET](#)
- [OpenAI SDK for JavaScript](#)

This exercise takes approximately **30** minutes.

Note: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- An active [Azure subscription](#)
- [Visual Studio Code](#) installed
- [Python version 3.13.xx](#) installed*
- [Git](#) installed and configured
- [Azure CLI](#) installed

* Python 3.14 is available, but some dependencies are not yet compiled for that release. The lab has been successfully tested with Python 3.13.12.

Create a Microsoft Foundry project

Microsoft Foundry uses projects to organize models, resources, data, and other assets used to develop an AI solution.

1. In a web browser, open the [Microsoft Foundry portal](#) at `https://ai.azure.com` and sign in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in, and if necessary use the Foundry logo at the top left to navigate to the home page.
2. If it is not already enabled, in the tool bar the top of the page, enable the **New Foundry** option. Then, if prompted, create a new project with a unique name; expanding the **Advanced options** area to specify the following settings for your project:
 - **Foundry resource:** Use the default name for your resource (usually {project_name}-resource)
 - **Subscription:** Your Azure subscription
 - **Resource group:** Create or select a resource group
 - **Region:** Select East US 2 (For this exercises some models are only available in this location.)
3. Select **Create**. Wait for your project to be created. Then view its home page.

Deploy models

To develop speech-enabled apps, we're going to need speech-enabled models. Specifically, we need a model that can perform speech-generation, and a model that can process speech input.

Deploy a speech-generation model

1. Now you're ready to **Start building**. Let's **Explore models** (or on the **Discover** page, select the **Models** tab) to view the Microsoft Foundry model catalog.
2. In the model catalog, search for `gpt-4o-mini-tts` or another text-to-speech model available in your region.
3. Review the model card, and then deploy an available text-to-speech model using the default settings.
4. When the model has been deployed, view its details, noting that the **Target URI** and **Key** required to use it are available here (you'll need the Target URI later).

! IMPORTANT: `gpt-4o-mini-tts` is a recommended model for this exercise, not a required one. If it is unavailable in your region, deploy a supported text-to-speech model instead.

Deploy a speech-recognition model

1. In the Foundry portal menu bar, select **Build**; and then view the **Deployments** page. Note that the text-to-speech model you deployed is listed.
2. Select **Deploy a base model**, and search the catalog for `gpt-4o-mini-transcribe` or another speech-to-text model available in your region.
3. Deploy an available speech-to-text model using the default settings.
4. Return to the **Deployments** page and verify that both of the model you deployed are listed.
5. Select either of the models to view the Target URI you need to use in your code.

! IMPORTANT: `gpt-4o-mini-transcribe` is a recommended model for this exercise, not a required one. If it is unavailable in your region, deploy a supported speech-to-text model instead.

Get the application files from GitHub

The initial application files you'll need to develop speech applications are provided in a GitHub repo.

1. Open Visual Studio Code.
2. Open the command palette (*Ctrl+Shift+P*) and use the `Git:clone` command to clone the `https://github.com/microsoftlearning/mslearn-ai-language` repo to a local folder (it doesn't matter which one). Then open it.

You may be prompted to confirm you trust the authors.

3. In Visual Studio Code, view the **Extensions** pane; and if it is not already installed, install the **Python** extension.
4. In the **Command Palette**, use the command `python:select interpreter`. Then select an existing environment if you have one, or create a new **Venv** environment based on your Python 3.1x installation.

! Tip: If you are prompted to install dependencies, you can install the ones in the *requirements.txt* file in the */Labfiles/03-gen-ai-speech/Python/generate-speech* folder; but it's OK if you don't - we'll install them later!

! **Tip:** If you prefer to use the terminal, you can create your **Venv** environment with `python -m venv labenv`, then activate it with `\labenv\Scripts\activate`.

Create a speech-generation app

1. After the repo has been cloned, in the Explorer pane, navigate to the folder containing the application code files at **/Labfiles/03-gen-ai-speech/Python/generate-speech**. The application files include:
 - **.env** (the application configuration file)
 - **requirements.txt** (the Python package dependencies that need to be installed)
 - **generate-speech.py** (the code file for the application)

Configure your application

1. In the **Explorer** pane, right-click the **generate-speech** folder containing the application files, and select **Open in integrated terminal** (or open a terminal in the **Terminal** menu and navigate to the */Labfiles/03-gen-ai-speech/Python/generate-speech* folder.)

! **Note:** Opening the terminal in Visual Studio Code will automatically activate the Python environment. You may need to enable running scripts on your system.

2. Ensure that the terminal is open in the **generate-speech** folder with the prefix **(.venv)** to indicate that the Python environment you created is active.
3. Install the OpenAI SDK package and other required packages by running the following command:

Code Copy

```
pip install -r requirements.txt
```

4. In the **Explorer** pane, in the **generate-speech** folder, select the **.env** file to open it. Then update the configuration values to include the **Target URI** (endpoint) for your **gpt-4o-mini-tts** model.

! **Tip:** Copy the Target URI from the model details page in the Foundry portal.

Save the modified configuration file.

Write code to use the model for speech-generation

1. In the **Explorer** pane, in the **generate-speech** folder, select the **generate-speech.py** file to open it.
2. Review the existing code. You will add code to use the OpenAI SDK to access your model.

! **Tip:** As you add code to the code file, be sure to maintain the correct indentation.

3. At the top of the code file, under the existing namespace references, find the comment **Import namespaces** and add the following code to import the namespace you will need to use the OpenAI SDK:

Code Copy

```
# import namespaces
from openai import AzureOpenAI
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
```

4. In the **main** function, note that code to load the endpoint from the configuration file has already been provided. Then find the comment **Create the Azure OpenAI client**, and add the following code to create a client for the OpenAI API:

CodeCopy

```
# Create the Azure OpenAI client
token_provider = get_bearer_token_provider(
    DefaultAzureCredential(), "https://ai.azure.com/.default"
)

client = AzureOpenAI(
    azure_endpoint=endpoint,
    azure_ad_token_provider = token_provider,
    api_version="2025-03-01-preview"
)
```

5. Find the comment **Generate speech and save to file**, and add the following code to submit a prompt to the speech-generation model save the response as a file.

CodeCopy

```
# Generate speech and save to file
with client.audio.speech.with_streaming_response.create(
    model=model_deployment,
    voice="alloy",
    input="My voice is my passport!",
    instructions="Speak in a serious tone.",
) as response:
    response.stream_to_file(speech_file_path)
```

6. Save the changes to the code file.

Run the application

1. In the terminal pane, use the following command to sign into Azure.

CodeCopy

```
az login
```

Note: In most scenarios, just using *az login* will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the *--tenant* parameter. See [Sign into Azure interactively using the Azure CLI](#) for details.

2. When prompted, follow the instructions to sign into Azure. Then complete the sign in process in the command line, viewing (and confirming if necessary) the details of the subscription containing your Foundry resource.

3. After you have signed in, enter the following command to run the application:

CodeCopy

```
python generate-speech.py
```

4. Observe the output as the code generates the requested speech and saves it in a file. The code should also play the generated audio file.

Create a speech-transcription app

1. In the Explorer pane, navigate to the folder containing the application code files at **/Labfiles/03-gen-ai-speech/Python/transcribe-speech**. The application files include:
- **.env** (the application configuration file)

○ **requirements.txt** (the Python package dependencies that need to be installed)

○ **transcribe-speech.py** (the code file for the application)

Configure your application

1. In the **Explorer** pane, right-click the **transcribe-speech** folder containing the application files, and select **Open in integrated terminal** (or in the existing terminal, navigate to the */Labfiles/03-gen-ai-speech/Python/transcribe-speech* folder.)

! **Note:** Opening the terminal in Visual Studio Code will automatically activate the Python environment. You may need to enable running scripts on your system.

2. Ensure that the terminal is open in the **transcribe-speech** folder with the prefix **(.venv)** to indicate that the Python environment you created previously is active.
3. Install the OpenAI SDK package and other required packages by running the following command:

Code

Copy

```
pip install -r requirements.txt
```

! **Note:** This step isn't actually necessary if you completed the previous part of this exercise, as botg apps use the same environment and have the same dependencies - but it won't do any harm!

4. In the **Explorer** pane, in the **transcribe-speech** folder, select the **.env** file to open it. Then update the configuration values to include the **Target URI** (endpoint) for your **gpt-4o-mini-transcribe** model.

! **Tip:** Copy the Target URI from the model details page in the Foundry portal.

Save the modified configuration file.

Write code to use the model for speech-transcription

1. In the **Explorer** pane, in the **transcribe-speech** folder, select the **transcribe-speech.py** file to open it.
2. Review the existing code. You will add code to use the OpenAI SDK to access your model.

! **Tip:** As you add code to the code file, be sure to maintain the correct indentation.

3. At the top of the code file, under the existing namespace references, find the comment **Import namespaces** and add the following code to import the namespace you will need to use the OpenAI SDK:

Code

Copy

```
# import namespaces
from openai import AzureOpenAI
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
```

4. In the **main** function, note that code to load the endpoint from the configuration file has already been provided. Then find the comment **Create the Azure OpenAI client**, and add the following code to create a client for the OpenAI API:

CodeCopy

```
# Create the Azure OpenAI client
token_provider = get_bearer_token_provider(
    DefaultAzureCredential(), "https://ai.azure.com/.default"
)

client = AzureOpenAI(
    azure_endpoint=endpoint,
    azure_ad_token_provider = token_provider,
    api_version="2025-03-01-preview"
)
```

5. Find the comment **Call model to transcribe audio file**, and add the following code to submit an audio file to the speech-transcription model generate a transcript.

CodeCopy

```
# Call model to transcribe audio file
audio_file = open(file_path, "rb")
transcription = client.audio.transcriptions.create(
    model=model_deployment,
    file=audio_file,
    response_format="text"
)

print(transcription)
```

6. Save the changes to the code file.

Run the application

1. In the terminal pane, use the following command to sign into Azure.

CodeCopy

```
az login
```

! **Note:** In most scenarios, just using *az login* will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the *--tenant* parameter. See [Sign into Azure interactively using the Azure CLI](#) for details.

2. When prompted, follow the instructions to sign into Azure. Then complete the sign in process in the command line, viewing (and confirming if necessary) the details of the subscription containing your Foundry resource.

3. After you have signed in, enter the following command to run the application:

Code

```
python transcribe-speech.py
```

Copy

4. Observe the output as the code submits the audio file to the model for transcription and displays the results. The code should also play the audio file.

Clean up

If you've finished exploring speech-enabled models in Foundry Tools, you should delete the resources you have created in this exercise to avoid incurring unnecessary Azure costs.

1. Open the [Azure portal](#) and view the contents of the resource group where you deployed the resources used in this exercise.
2. On the toolbar, select **Delete resource group**.
3. Enter the resource group name and confirm that you want to delete it.